

SwipeLab



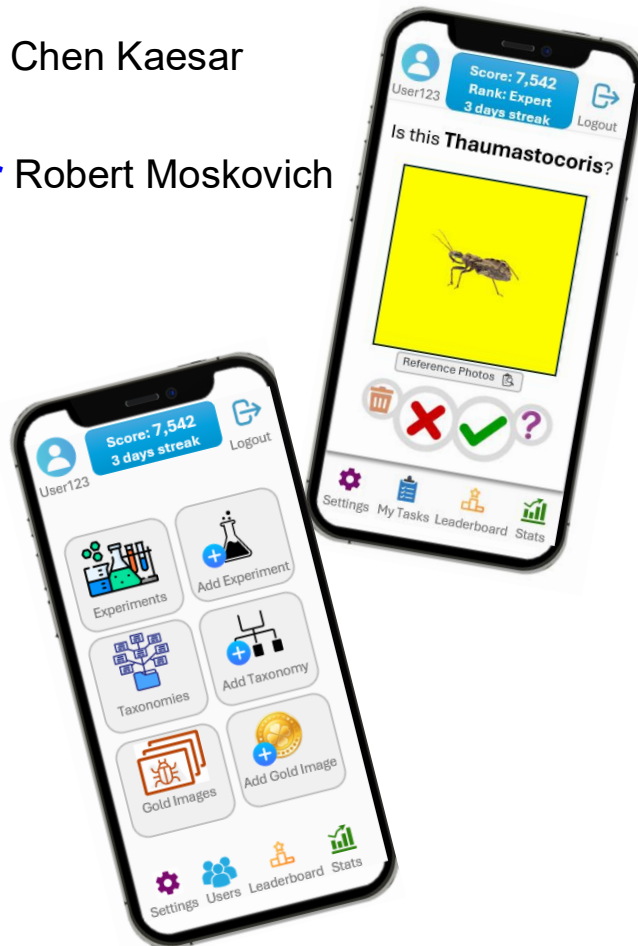
Application Design Document

Technical Advisor Chen Kaesar

Academic Advisor Robert Moskovich

Team Members

- Sagi Evroni
- Eden Bar
- Ofri Hanochi
- Avihoo Amos



SwipeLab	1
1. Use Cases	3
2. System Architecture	10
2.1 Overview	10
2.2 Subsystems and Components	10
2.3 Interfaces and Communication	11
2.4 Technology Stack	11
3. Data Model	12
3.1 Description of Data Objects	12
3.2 Data Objects Relationships	14
3.3 Databases	14
4. Behavioral Analysis	16
4.1 Sequence Diagrams	16
4.2 Events	22
4.3 States	23
5. Object-Oriented Analysis	24
5.1 Class Diagrams	24
5.2 Class Description	25
5.3 Packages	30
5.4 Unit Testing	31
6. User Interface Draft	33
7. Testing	42

1. Use Cases

UC-1: Register New Account

- **Primary Actor:** Volunteer (Guest User)
- **Description:** A new user creates an account to access the SwipeLab platform.
- **Trigger:** The user launches the app and taps "Sign Up" or "Create Account."
- **Preconditions:**
 1. The app is installed.
 2. The device has an active internet connection.
- **Postconditions:**
 1. A new user record is created in the database.
 2. The user is logged in or prompted to verify their email.
- **Main Success Scenario:**
 1. The user navigates to the registration screen.
 2. The user enters a **Username** (3-20 characters).
 3. The user enters a valid **Email Address**.
 4. The user creates a **Password** (min 8 characters).
 5. The user enters a **Display Name**.
 6. The user submits the form.
 7. The system validates the input and creates the account.
 8. The system confirms successful registration.

UC-2: Authenticate and Onboard

- **Primary Actor:** Volunteer (End User)
- **Description:** The user securely logs into the application and is shown a brief introduction to the project's purpose.
- **Trigger:** The user opens the application for the first time or after being logged out.
- **Preconditions:**
 1. The app is installed on the user's mobile device.
 2. The device has an active internet connection.
- **Postconditions:**
 1. The user is authenticated, and a secure session is created.
 2. The user's profile (for tracking points and credibility) is created or retrieved.
- **Main Success Scenario:**
 1. The user opens the app and is presented with the "Login / Onboarding" screen.
 2. The screen provides a short intro explaining the project's purpose and privacy.
 3. The user taps the "Google Sign-In" button.
 4. The system uses OAuth2 to securely authenticate the user with their Google account.
 5. The backend verifies the user's token and links them to their SwipeLab profile.
 6. The system proceeds to the main "Classification Screen" (UC-2).

UC-3: Select and Play Task

- **Primary Actor:** Volunteer (End User)
- **Description:** The user browses their assigned tasks, views details for a specific task, and begins the classification session.
- **Trigger:** The user navigates to the "My Tasks" dashboard.
- **Preconditions:**
 1. The user is logged in.
 2. At least one task is assigned to the user (or available publicly).
- **Postconditions:**
 1. The user enters the classification mode (Swipe Screen) for the selected task.
- **Main Success Scenario:**
 1. The user views the "My Tasks" page.
 2. The system displays a paginated list of available tasks.
 3. The user selects a specific task from the list.
 4. The system retrieves and displays the task details (e.g., description, progress).
 5. The user taps the "Play" or "Start" button.
 6. The system initializes a classification session for that task ID.

UC-4: Classify Image Stream

- **Primary Actor:** Volunteer (End User)
- **Description:** The user classifies a stream of images using simple swipe gestures to earn points and contribute data to the research project.
- **Trigger:** The user is logged in and is on the "Classification Screen".
- **Preconditions:**
 - The user is authenticated (UC-1).
- **Postconditions:**
 - The user's classifications are stored in the backend database.
 - The user's points and reliability score are updated.
- **Main Success Scenario:**
 - The system retrieves and displays an image from the STARdbi dataset API.
 - The user analyzes the image and swipes (or taps) "Yes," "No," or "I Don't Know" or "Trash".
 - The system provides immediate visual feedback for the action.
 - The system securely records the user's label and associated metadata (e.g., response time).
 - The system updates the user's points and progress bar.
 - The system serves the next image in the batch.
- **Alternative Flows:**
 - **2a. System serves a "Trap Image":**
 1. The system retrieves and displays a "gold-standard" image with a known label.
 2. The user provides a classification.
 3. The system compares the user's answer to the known label and updates the user's hidden "credibility score".

UC-5: View Personal Progress and Leaderboard

- **Primary Actor:** Volunteer (End User)
- **Description:** The user accesses their personal statistics and the community leaderboard to track their progress and ranking, which directly supports the project goal of increasing user engagement.
- **Trigger:** The user taps on the "Progress" or "Leaderboard" tab/button in the app.
- **Preconditions:**
 1. The user is logged in (UC-1).
- **Postconditions:**
 1. None (this is a view-only action).
- **Main Success Scenario (Personal Stats):**
 1. User navigates to the "Progress / Points Screen".
 2. The system retrieves and displays the user's personal statistics, including:
 - Total points earned.
 - Badges unlocked.
 - Credibility score/level.
 - Total classifications.
 - Accuracy vs. expert consensus.
 - Time spent and streak days.
- **Main Success Scenario (Leaderboard):**
 1. User navigates to the "Leaderboard" screen.
 2. The system retrieves and displays a list of top contributors, such as the "LabSwiper of the Week / Month" .

UC-6: View and Track Challenges

- **Primary Actor:** Volunteer (End User)
- **Description:** The user views available gamification challenges (e.g., Daily, Weekly, Streak) to understand their current objectives and progress.
- **Trigger:** The user taps on the "Challenges" tab.
- **Preconditions:**
 - The user is logged in.
- **Postconditions:**
 - User is aware of active goals.
- **Main Success Scenario:**
 1. The system fetches the list of challenges (Daily, Weekly, Streak, Milestone).
 2. The system calculates and displays the progress percentage for each challenge (e.g., "10/20 images classified").
 3. The system visually distinguishes between "In Progress" and "Completed" challenges.
 4. The user taps a challenge to view the specific reward (e.g., "Silver Badge").

UC-7: Add a New Classification Task

Primary Actor: Research Scientist (Administrator)

Description: The researcher creates a new classification task for a specific bug or organism type by uploading reference pictures and defining the classification question to guide mobile app users.

Trigger: The researcher navigates to the Research Dashboard and clicks "Create New Classification Task."

Preconditions:

1. The researcher has administrator-level credentials.
2. The researcher has prepared reference images showing the target organism.
3. At least one dataset exists or is being uploaded concurrently.

Postconditions:

1. A new classification task is created with reference pictures.
2. The task is associated with a dataset and available to mobile users.
3. Users can view reference pictures when classifying this task.

Main Success Scenario:

1. The researcher logs into the research dashboard.
2. The researcher clicks "Create New Classification Task."
3. The system displays the task creation form.
4. The researcher enters task details:
 - Task name (e.g., "Asian Giant Hornet Identification")
 - Classification question (e.g., "Is this an Asian Giant Hornet?")
 - Brief description of identifying features
5. The researcher uploads 3-10 reference pictures showing clear examples of the target organism.
6. For each reference picture, the researcher adds captions explaining key features.
7. The researcher optionally uploads negative examples (similar-looking species for comparison).
8. The researcher configures quality parameters (minimum classifications per image, consensus threshold).
9. The researcher reviews the configuration and clicks "Create Task."
10. The system validates the task configuration and stores the reference images.
11. The researcher associates the task with an existing dataset.
12. The system activates the task and makes it available to mobile app users.
13. When users first encounter this task, they see a tutorial displaying the reference pictures.

Alternative Flows:

- **5a.** No reference images uploaded: System warns but allows task creation if researcher confirms.
- **10a.** Validation fails (missing required information): System highlights issues and prevents creation until resolved.
- **11a.** No suitable dataset exists: System prompts researcher to upload dataset first.
- **13a.** User skips tutorial: Reference pictures remain accessible via info button during classification.

UC-8: Define Trap Pictures for User Reliability Grading

Primary Actor: Research Scientist (Administrator)

Description: The researcher defines "trap pictures" (also known as gold standard or quality control images) with known correct classifications. These images are inserted into user classification queues to measure accuracy and calculate user reliability scores.

Trigger: The researcher navigates to the SwipeLab "Research Dashboard" and selects an existing classification task to add quality control images.

Preconditions:

1. The researcher has administrator-level credentials.
2. A classification task has been created.
3. The researcher has expert-verified correct labels for selected images.
4. At least 10 images are available in the dataset to serve as trap pictures.

Postconditions:

1. Trap pictures are designated in the system with their correct classifications.
2. The system begins inserting trap pictures into user classification queues at the specified frequency.
3. User reliability scores can be calculated based on accuracy on trap pictures.
4. The researcher can monitor user performance against trap pictures in real-time.

Main Success Scenario:

1. The researcher logs into the research dashboard.
2. The researcher navigates to the classification task detail page.
3. The researcher clicks on the "Quality Control" or "Trap Pictures" tab.
4. The system displays all images in the dataset with their current classification status.
5. The researcher selects images to designate as trap pictures (recommended: 5-15% of total dataset).
6. For each selected trap picture, the researcher provides the correct classification (e.g., "Yes" - this is a bee, "No" - this is not a bee)
7. The researcher configures trap picture parameters:
 - Insertion frequency (e.g., 1 trap picture every 10-15 regular images)
 - Whether to provide immediate feedback to users after trap picture classification
 - Reliability score impact weight (how much each correct/incorrect answer affects user score)
 - Minimum number of trap pictures a user must encounter before reliability score is calculated
8. The researcher reviews the trap picture configuration and distribution settings.
9. The researcher clicks "Activate Trap Pictures."
10. The system validates that trap pictures represent diverse difficulty levels and image types.
11. The system saves the trap picture configuration and begins inserting them into active user queues.
12. The system displays a confirmation message: "Trap pictures activated. User reliability tracking is now enabled."
13. The researcher can view real-time reliability statistics on the dashboard.

UC-9: Manage Recipient Groups

- **Primary Actor:** Researcher (Administrator)
- **Description:** The admin creates a defined list of users (Recipient Group) that can be targeted for specific classification tasks (e.g., "Expert Annotators", "Biology Students").
- **Trigger:** The researcher navigates to the "Recipient Groups" management section.
- **Preconditions:**
 - The researcher has administrator privileges.
- **Postconditions:**
 - A new recipient group is saved in the database.
 - The group becomes available for assignment in the Task Editor.
- **Main Success Scenario:**
 1. The researcher selects "Create Recipient Group."
 2. The researcher enters a group name.
 3. The researcher selects or inputs a list of user identifiers (usernames or emails) to include in the group.
 4. The researcher saves the group.
 5. The system validates the users exist and stores the group.

UC-10: Edit Classification Task

- **Primary Actor:** Researcher (Administrator)
- **Description:** The researcher updates the configuration of an existing task, including modifying its description, target species, or assigned user groups.
- **Trigger:** The researcher selects an existing task and clicks "Edit."
- **Preconditions:**
 - The task exists.
 - The researcher has administrator privileges.
- **Postconditions:**
 - The task configuration is updated in the database.
- **Main Success Scenario:**
 1. The system loads the current task data.
 2. The researcher updates the **Task Name** or **Description**.
 3. The researcher adds or removes **Target Species** configurations.
 4. The researcher modifies the **Experiments** linked to the task.
 5. The researcher updates the **Recipient Groups** to change who receives the task.
 6. The researcher saves the changes.
 7. The system updates the task and applies the new rules immediately.

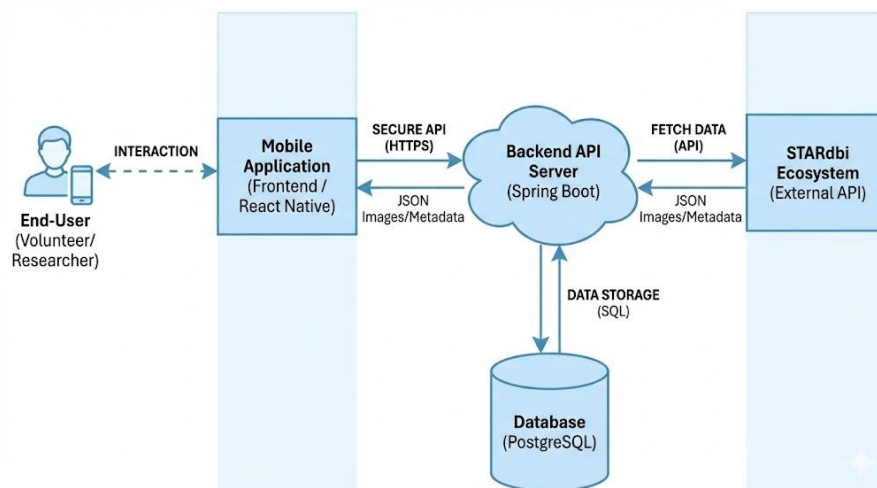
UC-11: Analyze and Export Data

- **Primary Actor:** Researcher (Administrator)
- **Description:** The researcher logs into the web dashboard to monitor the quality of the incoming labels and export a cleaned dataset for use in model training.
- **Trigger:** The researcher navigates to the SwipeLab "Research Dashboard" and logs in.
- **Preconditions:**
 1. The researcher has administrator-level credentials.
 2. Volunteers have submitted a sufficient number of classifications.
- **Postconditions:**
 1. The researcher receives a downloadable file (e.g., CSV, JSON) containing high-quality labeled data.
- **Main Success Scenario:**
 1. The researcher logs into the research dashboard.
 2. The system displays an overview of statistics, user stats, and consensus levels.
 3. The researcher navigates to the "Export Data" section.
 4. The researcher applies filters, such as a minimum "user reliability score," to exclude low-quality data.
 5. The researcher clicks "Generate Export."
 6. The system queries the database, filters the labels, and generates a file in the selected format.
 7. The researcher downloads the file.

2. System Architecture

2.1 Overview

The SwipeLab system follows a classic **Client-Server Architecture**, designed to support mobile-first data collection while maintaining centralized data management and analysis. The system is composed of three primary subsystems: the **Mobile Application (Frontend)**, the **Backend API Server**, and the **Database**. Additionally, the system integrates with the external **STARdbi** infrastructure to retrieve raw insect imagery.



2.2 Subsystems and Components

2.2.1 Mobile Application (Frontend)

The frontend is a cross-platform mobile application built using **React Native (Expo)**, ensuring compatibility with both iOS and Android devices. It serves as the primary interface for volunteers ("Citizen Scientists") and includes the Research Dashboard for administrators.

- **Responsibilities:**

- **User Interface:** Renders the "Tinder-style" swipe interface for image classification.
- **Gamification Engine:** Displays points, badges, and leaderboards to motivate users.
- **Local Persistence:** Caches images and classification queues to support offline functionality.
- **Admin Dashboard:** Provides researchers with screens to manage tasks, upload gold images, and view analytics (implemented via shared React Native screens).

2.2.2 Backend API Server

The backend is a monolithic application built with **Java Spring Boot**, structured into a layered architecture (Controller, Service, Repository). It acts as the central orchestrator for the system.

- **Responsibilities:**
 - **API Gateway:** Exposes secure RESTful endpoints for the mobile client.
 - **Authentication & Security:** Manages user identity using **OAuth2 (Google)** and **JWT** for session management.
 - **Business Logic:** Executes core algorithms for **consensus calculation** (aggregating user votes), **credibility scoring** (weighting users based on accuracy), and **fraud detection**.
 - **Integration:** Fetches raw data from the external STARdbi API and serves it to users.

2.2.3 Data Storage (Database)

The system uses **PostgreSQL** as its primary relational database management system (RDBMS).

- **Responsibilities:**
 - Persists structural data including **Users, Tasks, Images, Classifications,** and **Gamification Stats** (Badges, Points).
 - Ensures data integrity and transactional consistency for classification submissions.

2.2.4 External Systems (STARdbi)

The system relies on the **STARdbi Ecosystem** as the authoritative source of raw data.

- **Interaction:** SwipeLab pulls batches of insect trap scans and metadata from the STARdbi Dataset API.
 - **Data Flow:** Unlabeled images flow *from* STARdbi *to* SwipeLab; Aggregated, labeled datasets are exported *from* SwipeLab *back* to researchers.
-

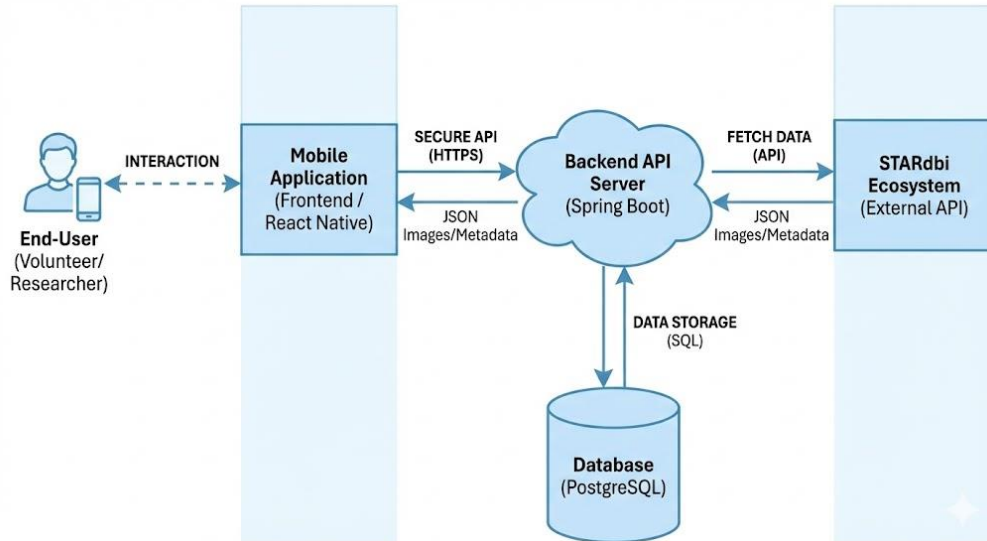
2.3 Interfaces and Communication

- **Client-Server Communication:** The Mobile App communicates with the Backend Server via **HTTPS** using **RESTful API** standards. All data is exchanged in **JSON** format.
 - **Authentication:** Communication is secured using **JWT (JSON Web Tokens)** included in the HTTP Authorization header.
 - **External API:** The Backend Server communicates with the STARdbi API via HTTP requests to fetch image URLs and metadata.
-

2.4 Technology Stack

- **Frontend Framework:** React Native (Expo), TypeScript.
- **Backend Framework:** Java Spring Boot.
- **Database:** PostgreSQL.
- **Security:** Spring Security, OAuth2, JWT.

- **Cloud/Infrastructure:** AWS S3 (or similar) for image storage integration.
- **Figure 2.1:** SwipeLab system architecture, detailing the secure communication between the Mobile Application and Backend API Server, as well as the backend's integration with the database and the external STARdbi ecosystem.



3. Data Model

3.1 Description of Data Objects

The SwipeLab data model consists of the following core entities that represent the system's state and business logic.

1. User

Represents a registered user (Volunteer or Researcher) in the system.

- **Key Attributes:**
 - username (Primary Key): Unique identifier.
 - email: User's contact email.
 - role (UserRole): Determines permissions (e.g., USER, ADMIN).
 - credibilityScore (Double): A calculated score (0.0 - 100.0) reflecting the user's reliability based on gold standard images.
 - points (Long): Gamification currency earned by classifying.
 - currentStreak (Integer): Consecutive days of activity.
 - authProvider (AuthProvider): Indicates if the user logged in via Local auth or OAuth (Google).

2. Task

Represents a specific classification assignment created by a researcher.

- **Key Attributes:**

- title & description: Metadata describing the scientific goal.
- status (TaskStatus): Lifecycle state (DRAFT, ACTIVE, PAUSED, ARCHIVED).
- minClassificationsPerImage: The number of votes needed per image to consider it "done".
- consensusThreshold: The percentage of agreement required (e.g., 80%).

3. Image

Represents a single visual data point to be classified.

- **Key Attributes:**

- imageUrl: URL pointing to the image storage (e.g., S3).
- isGoldStandard (Boolean): Flags if this is a "trap" image with a known answer for quality control.
- priority: Determines the order in which images are served to users.

4. Classification

Represents a single vote cast by a user on an image.

- **Key Attributes:**

- timestamp: When the swipe occurred.
- **Associations:** Links a specific User to a specific Image with a chosen Label.

5. Label

Represents a taxonomic category or answer option (e.g., "Wasp", "Not Wasp").

- **Key Attributes:**

- name: The scientific or display name.
- description: Additional context for the user.

6. RecipientGroup

Represents a defined cohort of users (e.g., "Biology Class A") that can be assigned private tasks.

- **Key Attributes:**

- name: Unique name for the group.
- users: A set of users belonging to this group.

7. Badge

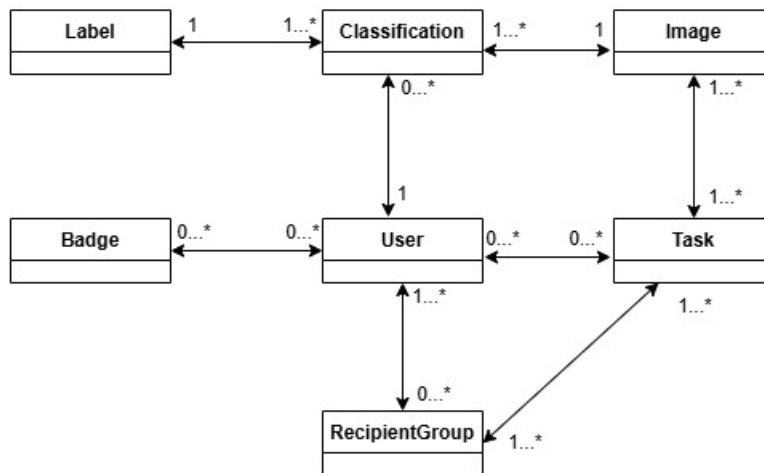
Represents a gamification achievement.

- **Key Attributes:**

- name: The display title of the achievement.
 - criteriaJson: A JSON structure defining the rules to unlock this badge.
 - iconUrl: Visual asset for the UI.
-

3.2 Data Objects Relationships

The entities are related as follows:



3.3 Databases

The system uses **PostgreSQL** as its Relational Database Management System (RDBMS).

- **Justification:** PostgreSQL was chosen for its robustness, strong support for complex queries (needed for consensus algorithms and analytics), and reliability in handling relational data structures.
- **Schema Generation:** The database schema is managed via **Flyway** migrations (as seen in `src/main/resources/db/migration`), ensuring version control and consistency across development and production environments.

Table Descriptions

The database consists of the following primary tables:

- **users** Stores all registered user accounts, including authentication credentials (password hash or OAuth IDs), profile details, and gamification statistics (points, credibility score, streaks).
- **tasks** Contains the definitions of classification assignments, including title, description, configuration parameters (consensus thresholds), and lifecycle status (DRAFT, ACTIVE, etc.).
- **images** Stores references to the insect images. It includes the image URL, priority level, and flags indicating if it is a "Gold Standard" (trap) image for quality control.
- **classifications** Records the individual votes cast by users. Each row links a user, an image, and a chosen label, acting as the central transactional record of the system.
- **labels** Defines the taxonomic categories (e.g., specific insect species or families) that users can select as answers.
- **badges** Contains the definitions of available gamification achievements, including their display names, icons, and the JSON logic required to unlock them.
- **recipient_groups** Stores named cohorts of users (e.g., "Biology Students 2024") created by administrators for targeted task distribution.

Join Tables

The following tables handle many-to-many relationships:

- **user_badges**: Maps users to the badges they have earned.
- **recipient_group_users**: Maps individual users to specific recipient groups.
- **task_target_species**: Links tasks to the specific labels (species) relevant for that assignment.
- **task_recipient_groups**: Links tasks to the recipient groups that are authorized to perform them.

Data Transactions

The SwipeLab system utilizes **ACID-compliant transactions** (Atomic, Consistent, Isolated, Durable) managed by the Spring Framework's declarative transaction management (@Transactional). This ensures that complex business operations involving multiple database tables either complete fully or roll back entirely, preventing data corruption.

The system defines the following critical transactional boundaries:

1. Classification Submission (Critical)

When a user swipes on an image, the system performs multiple updates that must occur atomically:

1. **Record Classification**: A new row is inserted into the classifications table.
2. **Update User Stats**: The user's total_classifications count is incremented.
3. **Award Points**: The user's points balance is updated in the users table.
4. **Update Streak**: If applicable, the user's current_streak and last_streak_update fields are modified.
5. **Check Consensus**: If the image reaches the consensus threshold, its status is updated.

Constraint: If any step fails (e.g., database connection loss during point calculation), the entire transaction is rolled back. The user's swipe is not recorded, and no partial points are awarded, ensuring the gamification state remains consistent with the scientific data.

2. User Registration

The onboarding process involves initializing several data entities simultaneously:

1. **Create User**: A new record is saved in the users table.
2. **Initialize Profile**: Default settings and an initial "neutral" credibility score are assigned.
3. **Token Generation**: For local auth, verification tokens are generated and stored.

3. Task Management (Admin)

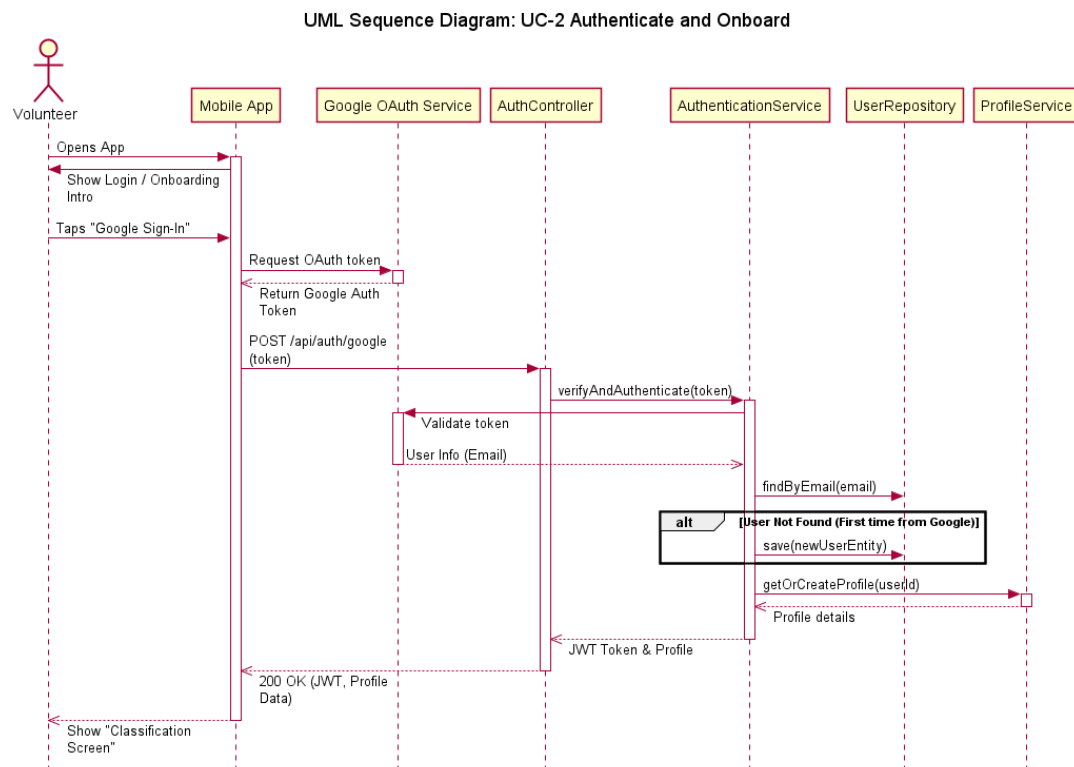
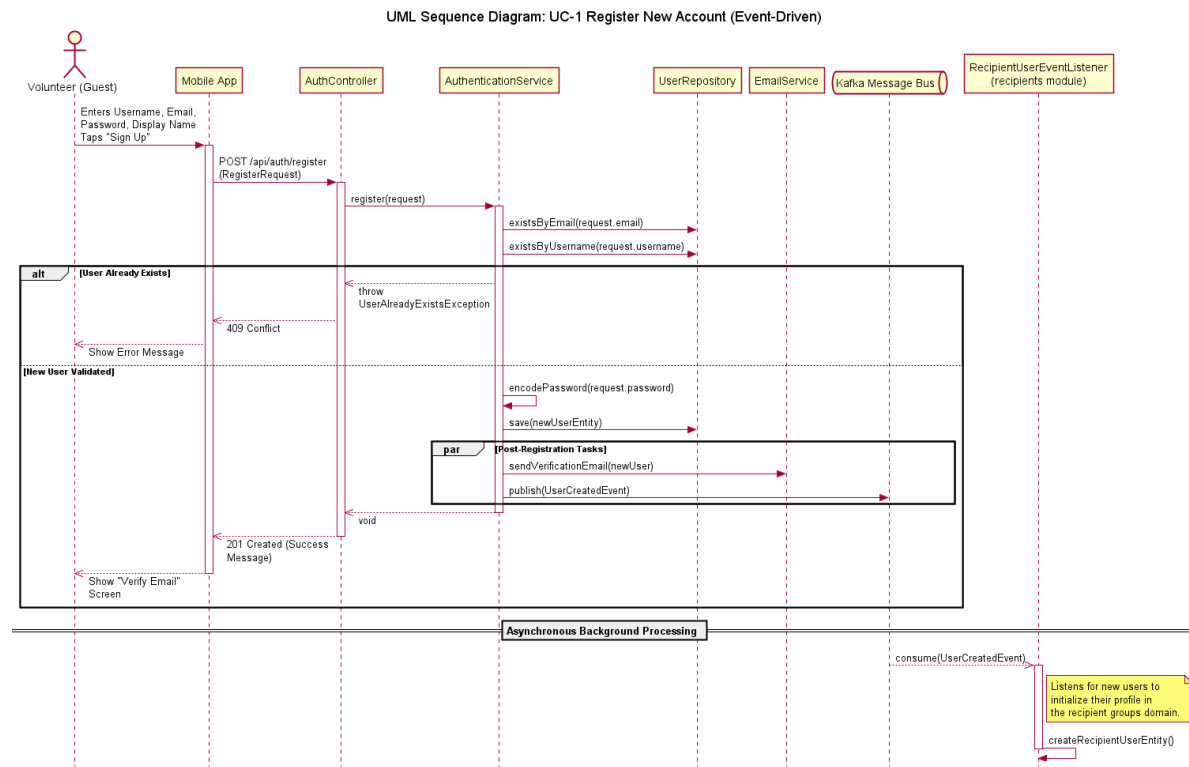
When a researcher creates or modifies a task:

1. **Save Task**: The Task entity is persisted.
2. **Link Images**: Associations between the task and potentially thousands of Image records are established.
3. **Update Metadata**: Target species (Labels) and recipient groups are linked.

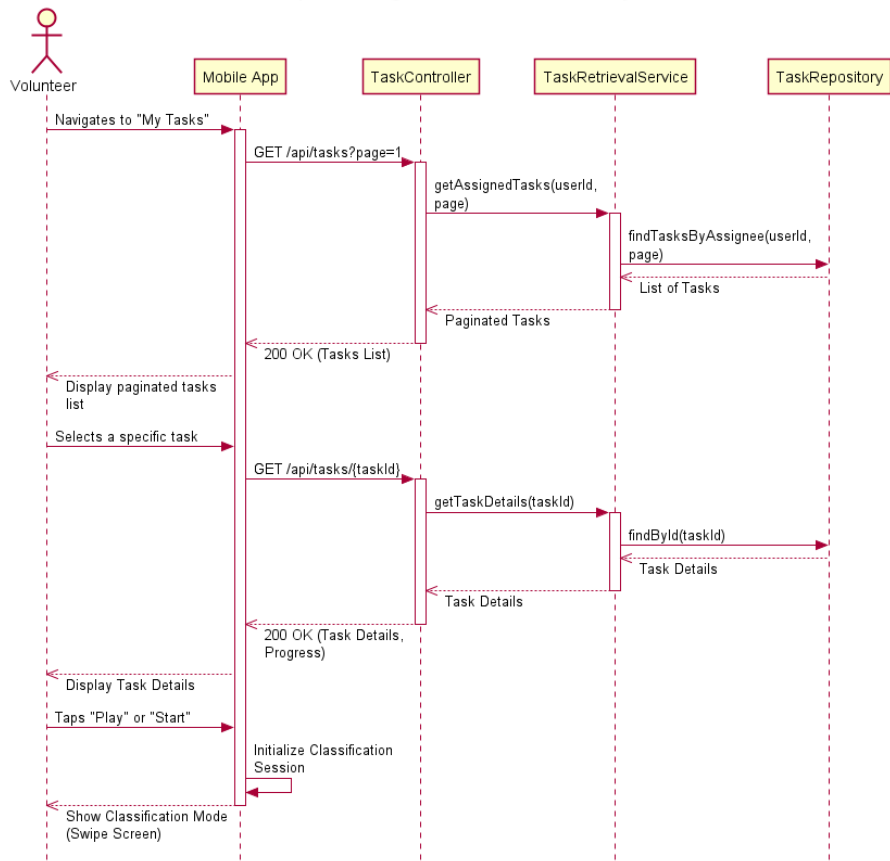
Constraint: This prevents "orphan" tasks that exist without their required configuration or images.

4. Behavioral Analysis

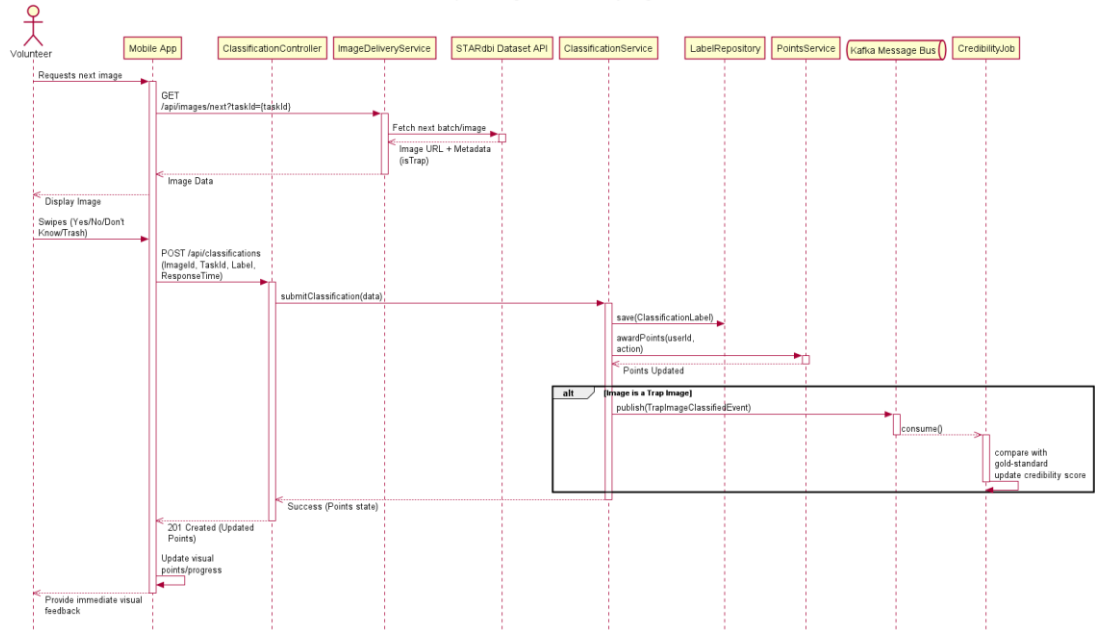
4.1 Sequence Diagrams



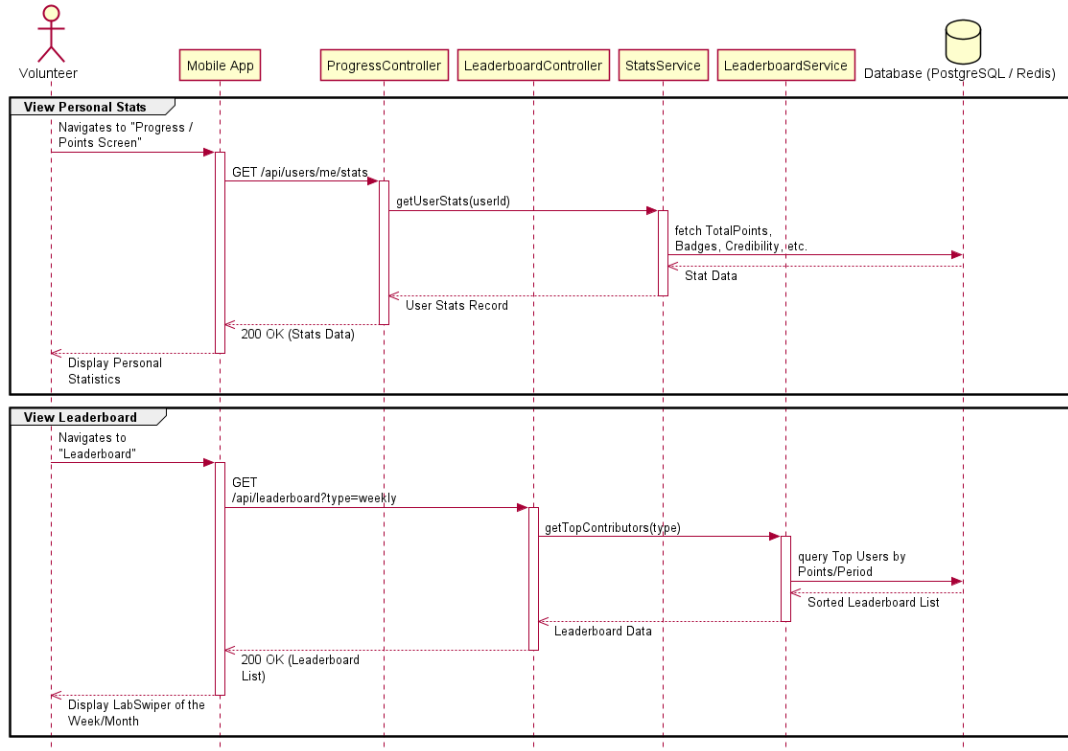
UML Sequence Diagram: UC-3 Select and Play Task



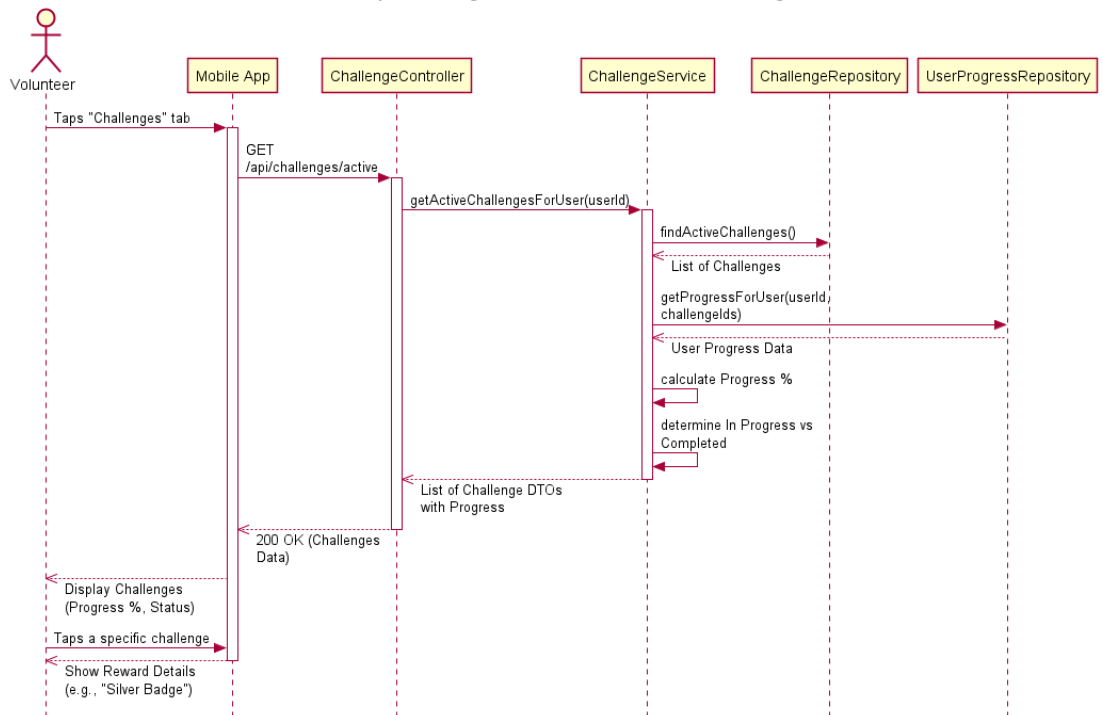
UML Sequence Diagram: UC-4 Classify Image Stream



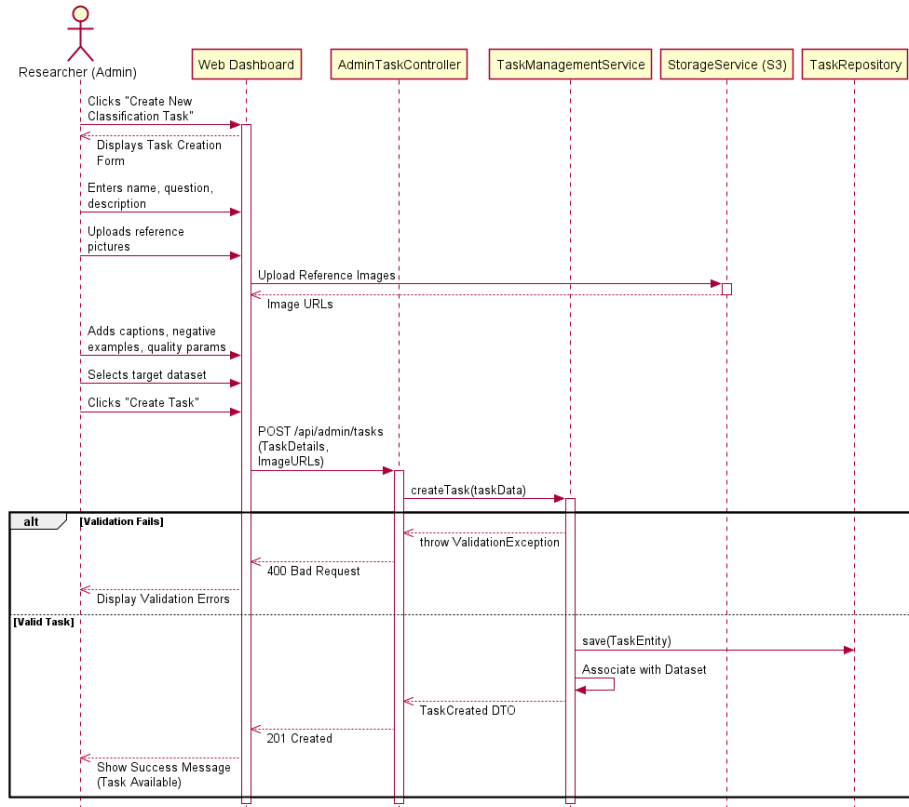
UML Sequence Diagram: UC-5 View Personal Progress and Leaderboard



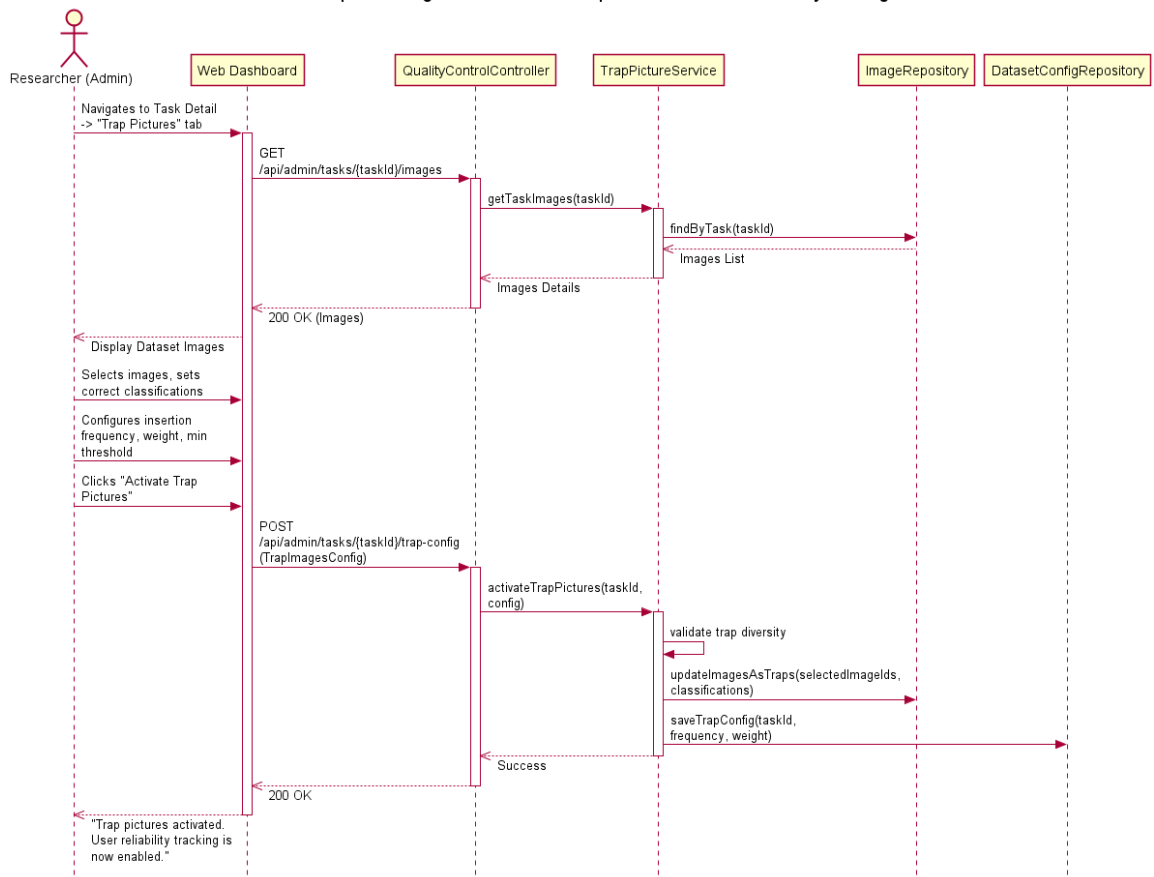
UML Sequence Diagram: UC-6 View and Track Challenges



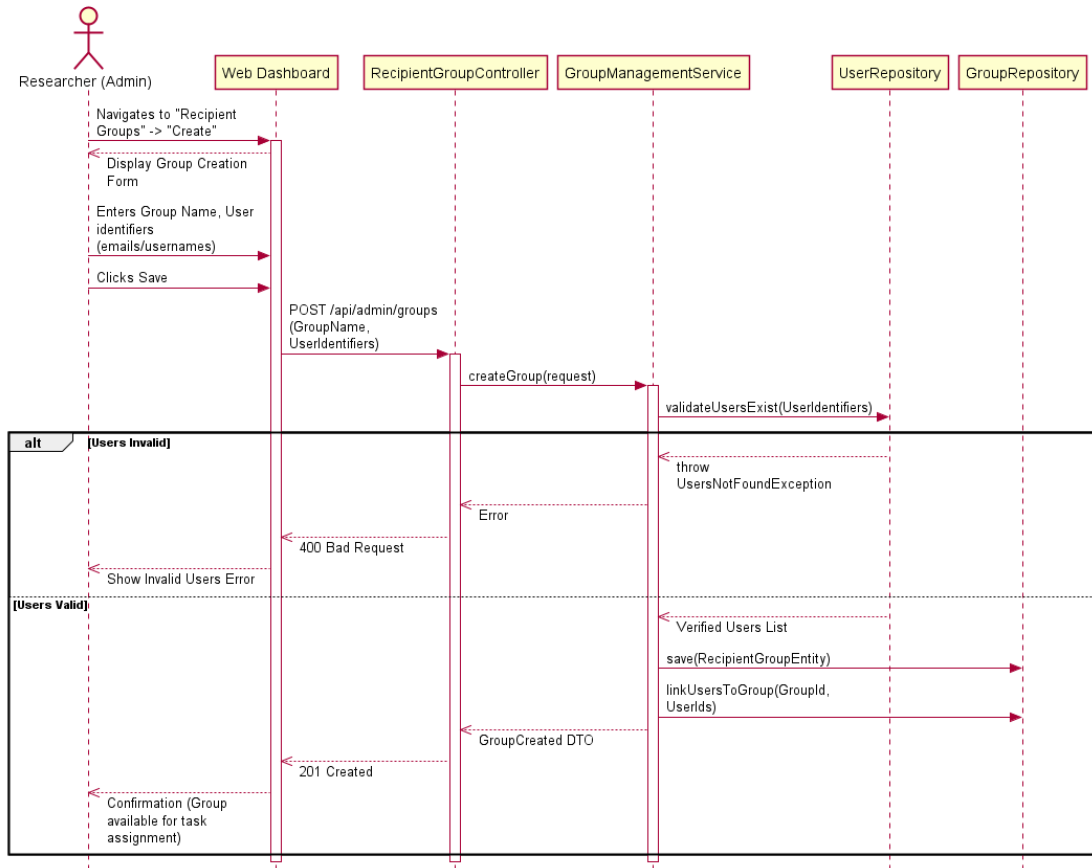
UML Sequence Diagram: UC-7 Add a New Classification Task



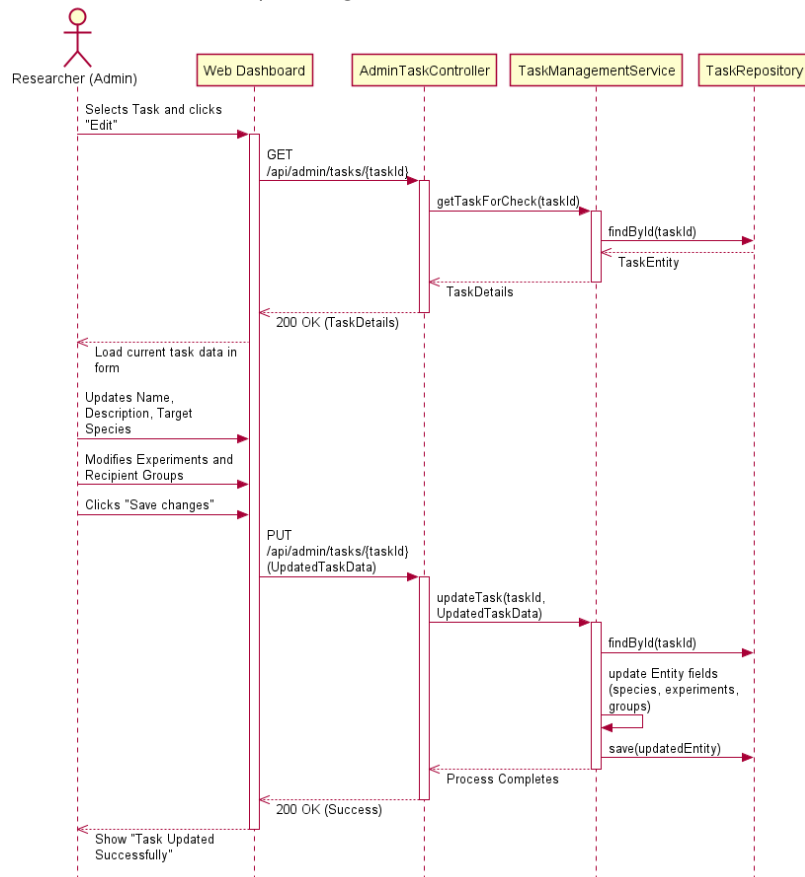
UML Sequence Diagram: UC-8 Define Trap Pictures for User Reliability Grading



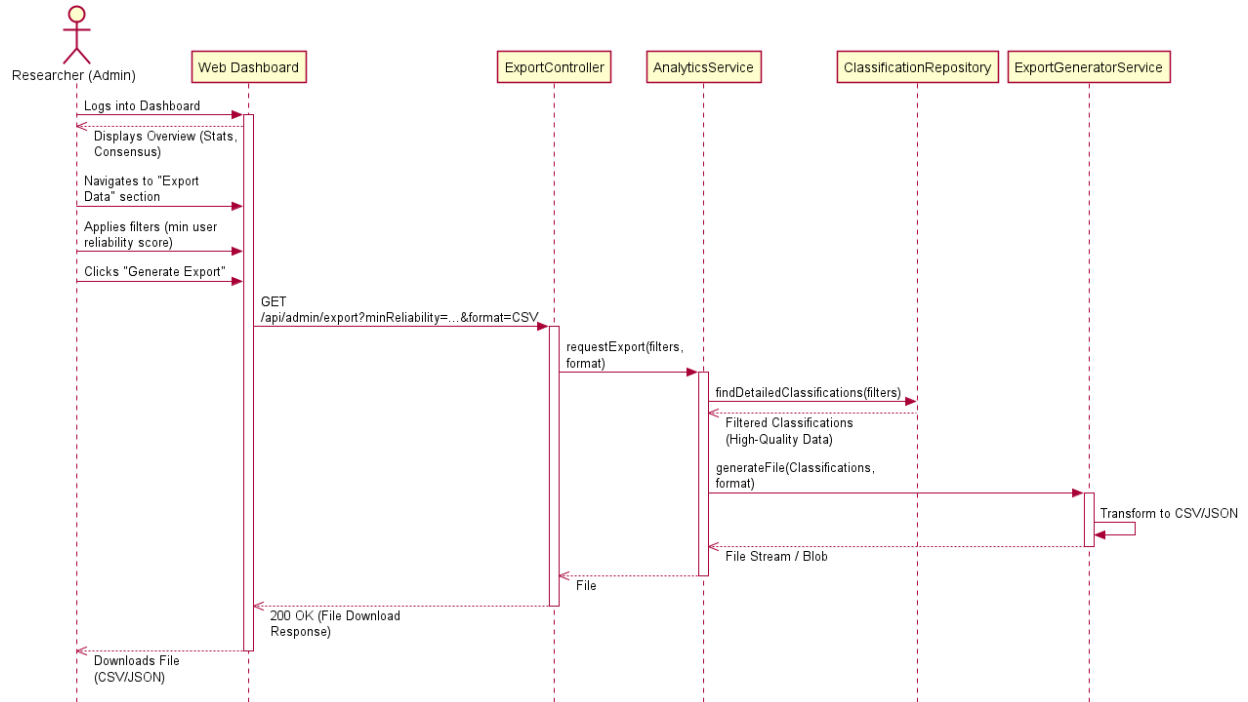
UML Sequence Diagram: UC-9 Manage Recipient Groups



UML Sequence Diagram: UC-10 Edit Classification Task



UML Sequence Diagram: UC-11 Analyze and Export Data

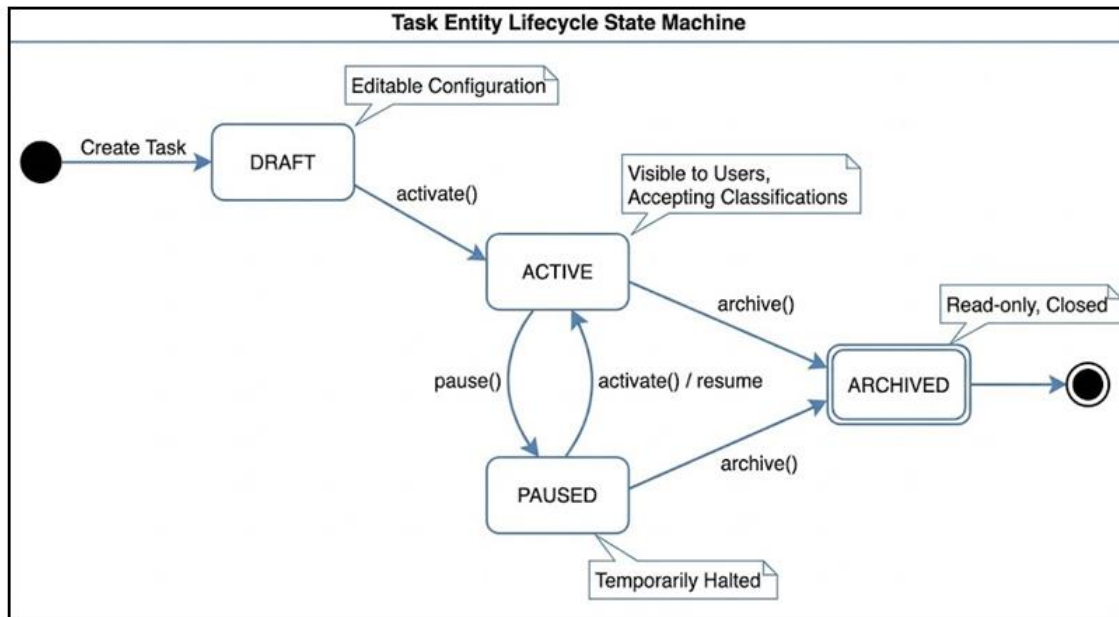


4.2 Events

Name	Description	Action
User Registration	End-User submits the registration form with email and password.	Validates input, creates a new USER record in the database, hashes the password, and triggers an email verification flow.
User Login	End-User submits credentials to access the system.	Authenticates credentials against the database. If successful, it generates and returns a JWT token for session management.
Select Task	End-User selects a specific task from the dashboard to start working.	Retrieves the task details, loads the next batch of unclassified Images (excluding those already seen), and initializes the session.
Classify Image	End-User swipes (left/right) or selects a label for an image.	Saves the Classification record. Triggers Consensus Service to check agreement, updates user's Points and Streak, and checks for new Badges.
View Analytics	End-User navigates to their profile or stats screen.	Aggregates and returns the user's performance metrics: accuracy, total swipes, current streak, and leaderboard ranking.
Create Task	Researcher defines a new task (title, description, target species).	Creates a new Task entity with status DRAFT. Associates selected target labels and configurations to the task.
Edit Task	Researcher modifies parameters of an existing task (e.g., consensus threshold).	Validates that the task is not in a restricted state (like Archived) and updates the configuration in the database.
Export Data	Researcher requests the results of a specific task.	Queries all classifications for the task, formats the data (CSV/JSON), and triggers a file download containing user judgments and consensus results.
Close Task	Researcher decides to stop data collection for a task.	Updates the TaskStatus to ARCHIVED or PAUSED, preventing end-users from fetching new images for this task.
Upload Gold Image	Researcher uploads an image with a known "correct" label.	Saves the image and links it to a GoldImage entity, defining the "Ground Truth" used to test user accuracy later.

4.3 States

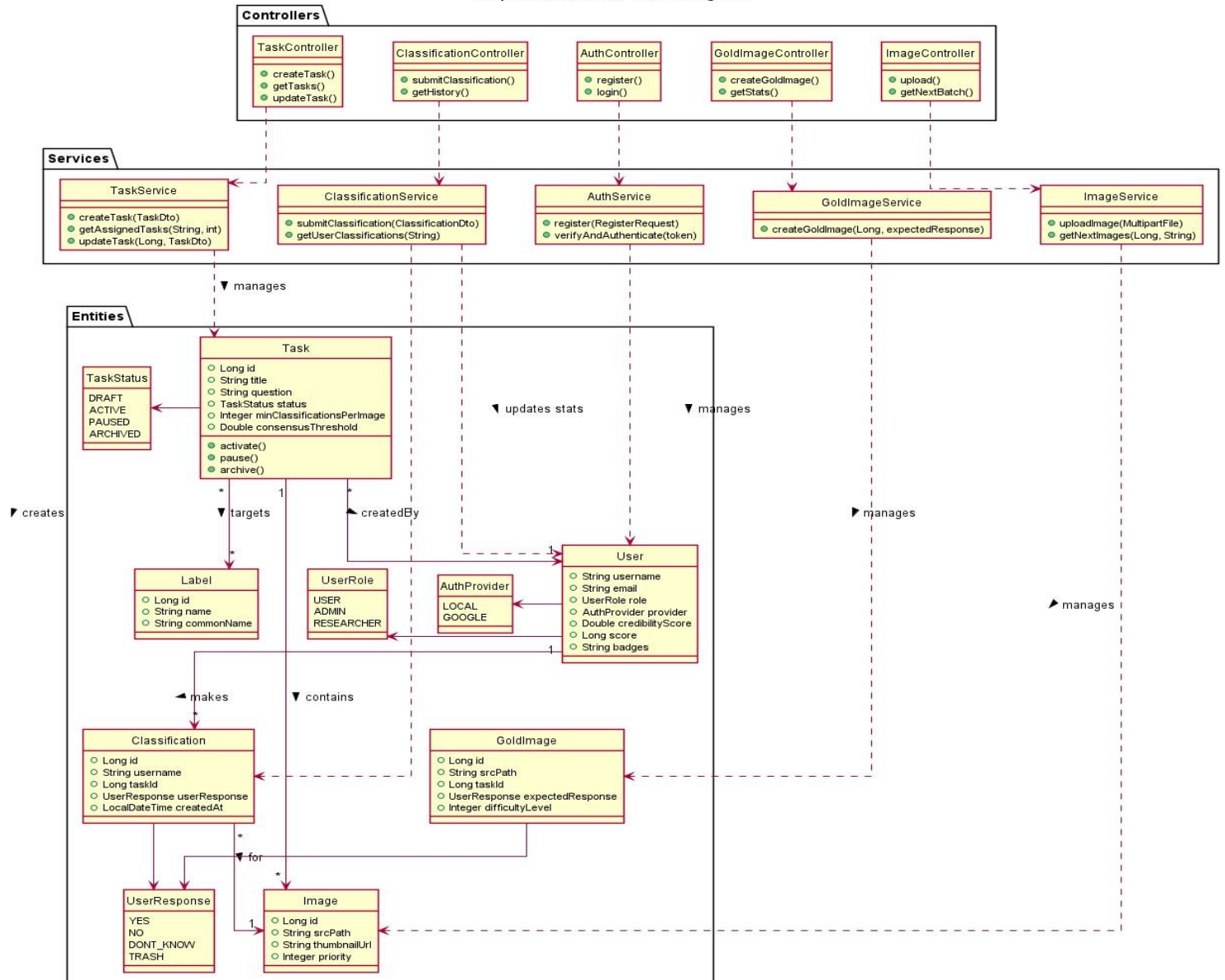
state machine for the **Task** entity, depicting the allowable transitions between its lifecycle stages – Draft, Active, Paused, and Archived – as enforced by the system's domain logic.



5. Object-Oriented Analysis

5.1 Class Diagrams

SwipeLab Backend Class Diagram



5.2 Class Description

1. Class: Task

Type: Entity (Domain Model - tasks module)

Description: Represents a classification assignment created by a researcher. It contains the configuration for the scientific experiment and manages its own lifecycle state (Draft, Active, Paused, Archived). It is now isolated within its own domain, completely decoupled from the core User profile.

- **Main Fields:**
 - status (TaskStatus): The current state of the task (e.g., DRAFT, ACTIVE).
 - minClassificationsPerImage (Integer): The number of independent user votes required before an image is considered "finished".
 - consensusThreshold (Double): The percentage of agreement required to validate a label (default 80.0).
 - targetSpecies (List<Label>): The specific insects or objects users are asked to identify.
- **Invariants:**
 - minClassificationsPerImage must be greater than 0.
 - consensusThreshold must be between 0.0 and 100.0.
- **Method: activate()**
 - **Pre-condition:** status is either DRAFT or PAUSED.
 - **Post-condition:** status is updated to ACTIVE.
- **Method: archive()**
 - **Pre-condition:** status is either ACTIVE or PAUSED.
 - **Post-condition:** status is updated to ARCHIVED.

2. Class: User

Type: Entity (Domain Model - users module)

Description: Represents a registered volunteer or researcher. It stores personal details, authentication credentials, and gamification progress (scores, streaks). *Update:* To support asynchronous eventing, gamification fields are now updated in the background rather than synchronously.

- **Main Fields:**
 - username (String): Unique identifier for the user.
 - credibilityScore (Double): A calculated value representing the reliability of the user's classifications.

- points (Long): Accumulated gamification currency earned by completing tasks.
- currentStreak (Integer): The number of consecutive days the user has been active.
- isFlagged (Boolean): *(New)* A security flag used by the async fraud detection system to suspend accounts.
- **Invariants:**
 - email must be unique across the system.
 - points cannot be negative.

3. Class: ClassificationService

Type: Service (Application Logic - classification module)

Description: Orchestrates the core "swiping" workflow. *Update:* This service has been refactored for high-throughput. It no longer synchronously calculates points or calls external domains. Instead, it validates the submission and publishes an event to Kafka.

- **Main Dependencies:** ClassificationRepository, KafkaEventPublisher, ValidationService *(Replaced PointsService & StreakService)*.
- **Method: submitClassification(String username, ClassificationRequest request)**
 - **Pre-conditions:**
 - username corresponds to a valid, existing User.
 - request.imageId corresponds to a valid Image.
 - request.labelId corresponds to a valid Label.
 - The user has not already classified this specific image.
 - **Post-conditions:**
 - A new Classification record is saved to the database.
 - *(New)* A ClassificationSubmittedEvent is published to the Kafka message bus.
 - *(New)* The API immediately returns 200 OK to the client.
 - *(Async Post-conditions handled by Kafka Consumers):* Gold Standard evaluation, totalClassifications increment, point addition, and streak updates happen strictly in the background.

4. Class: AuthenticationService

Type: Service (Application Logic - auth module)

Description: Manages the user registration and login processes, including security checks, JWT token generation, and new OAuth2 social login flows.

- **Main Dependencies:** UserRepository, JwtService, AuthenticationManager, KafkaEventPublisher.

- **Method: register(RegisterRequest request)**
 - **Pre-conditions:**
 - request.email does not already exist in the database.
 - request.username does not already exist in the database.
 - **Post-conditions:**
 - A new User is created with role = USER and provider = LOCAL.
 - A JWT access token and refresh token are generated and returned.
 - (New) A UserCreatedEvent is published to Kafka to initialize the user's statistics in downstream modules.
- **Method: verifyEmail(String token)**
 - **Pre-conditions:** The token is valid, matches a user's emailVerificationToken, and has not expired.
 - **Post-conditions:** The user's emailVerified status is set to true, and the token is cleared.

5. Class: AnalyticsService

Type: Service (Application Logic - analytics module)

Description: Responsible for aggregating raw classification data into actionable insights for researchers. *Update:* To optimize database performance, this service now queries materialized views and pre-calculated tables (TaskDailyStats) managed by Kafka listeners, rather than calculating everything on-the-fly.

- **Main Dependencies:** TaskDailyStatsRepository, UserRankingRepository (*Replaced direct domain repos*).
- **Method: getTaskAnalytics(Long taskId)**
 - **Description:** Aggregates all classifications for a task to calculate the overall consensus percentage, completion rate, and label distribution.
 - **Post-condition:** Returns a TaskAnalyticsResponse containing counts of "Low Consensus" (< 60%) vs "High Consensus" (> 80%) images.
- **Method: getUserPerformanceMetrics(Long taskId)**
 - **Description:** Generates a performance report for all users who contributed to a specific task.
 - **Logic:** Sorts users based on their goldAccuracy (percentage of correct calls on trap images).

6. Class: PointsService

Type: Service (Gamification)

Description: Manages the economic logic of the gamification system. *Update:* It is no longer directly coupled to the swiping flow. It is now called asynchronously by the GamificationOrchestratorService upon receiving a Kafka event.

- **Method: calculateAndAddPoints(User user, int basePoints)**
 - **Logic:** Applies a multiplier to basePoints based on the user's currentStreak:
 - Streak $\geq$ 30 days: 1.5x multiplier.
 - Streak $\geq$ 14 days: 1.25x multiplier.
 - Streak $\geq$ 7 days: 1.1x multiplier.
 - **Post-condition:** The user's total points balance is updated in the database.

7. Class: Badge

Type: Entity (Gamification)

Description: Represents a specific achievement or milestone that users can unlock (e.g., "Bee Expert", "Early Bird"). These criteria are evaluated asynchronously by the BadgeService listener.

- **Main Fields:**
 - name (String): Unique identifier for the badge.
 - criteriaJson (String): A JSON structure defining the rules required to unlock the badge (allows for flexible, dynamic criteria without changing the schema).
 - imageUrl (String): Visual asset for the UI.

8. Class: Image

Type: Entity (Domain Model - classification module)

Description: Represents the core unit of work in the system—a single photo requiring classification. It can be a standard sample or a "Gold Standard" image used for quality control.

- **Main Fields:**
 - imageUrl (String): Location of the file storage.
 - isGoldStandard (Boolean): Flag indicating if this is a "trap" image with a known answer.
 - correctLabel (Label): The ground-truth answer (Required if isGoldStandard is true).
 - priority (Integer): Determines the likelihood of this image being served to users earlier than others.
 - status (Enum): (*New*) Tracks if the image is PENDING, IN_PROGRESS, or COMPLETED based on real-time consensus.

9. Class: ImageService

Type: Service (Application Logic - classification module)

Description: Manages the lifecycle of images, including ingestion (upload) and distribution (batching) to users.

- **Method: uploadImage(ImageUploadRequest request)**
 - **Pre-condition:** If isGoldStandard is true, a correctLabelId must be provided.
 - **Post-condition:** Returns the created ImageResponse.
- **Method: getImageBatch(Long taskId, String username)**
 - **Logic:** Retrieves images for the task, filters out any images the specific username has already classified, shuffles the remaining set, and returns a limited batch (e.g., 20 images). *Update:* Also filters out images with a status of COMPLETED.
 - **Invariant:** A user should never receive an image they have already seen (unless explicitly reviewing).

10. Class: TaskService (Formerly part of AdminDashboardService) **Type:** Service (Application Logic - tasks module) **Description:** Handles the administrative lifecycle of tasks, refactored out of the monolithic AdminDashboardService to adhere to DDD.

- **Method: createTask(CreateTaskRequest request)**
 - **Post-condition:** A new task is persisted with status ACTIVE and is immediately available for distribution.

11. Class: RecipientGroupService (Formerly part of AdminDashboardService)

Type: Service (Application Logic - recipients module)

Description: Handles cohort creation, refactored out of the monolithic AdminDashboardService to strictly separate "Users" from "Assigned Groups".

- **Method: createRecipientGroup(CreateRecipientGroupRequest request)**
 - **Purpose:** creates a named cohort of users (e.g., "Biology Students 2024") that can be assigned specific private tasks.
 - **Invariant:** Group name must be unique.

(New Architecture Expansion Classes)

12. Class: ConsensusService

Type: Service (Domain Logic - classification module)

Description: The background engine responsible for aggregating votes. Consumes Kafka events to recalculate consensus using a weighted algorithm where user credibilityScore determines vote weight. Automatically transitions Image states to COMPLETED when the task's consensusThreshold is reached.

13. Class: GamificationOrchestratorService

Type: Service (Application Logic - gamification module)

Description: The centralized Kafka listener for all gamification logic. It decouples the core API from the reward system. Upon consuming a `ClassificationSubmittedEvent`, it dictates the execution order of the `PointsService`, `StreakService`, and `BadgeService`.

14. Class: `CredibilityService`

Type: Service (Application Logic - users module)

- **Description:** Extracts the "Gold Standard" evaluation logic out of the main swiping flow. It runs asynchronously, evaluating a user's submitted label against the `Image.correctLabel` and dynamically adjusting their `credibilityScore`.
-

5.3 Packages

5.3.1 Backend Packages

The SwipeLab backend is organized into a modular package hierarchy rooted at `com.swipelab`. Unlike the previous structure, the current architecture groups code by business domain to improve maintainability and scalability.

Root Package: `com.swipelab` Contains the main application entry point, `SwipeLabApplication.java`, which initializes the Spring Boot context and enables asynchronous processing.

1. **Domain-Driven Modules** Each package below represents a self-contained slice of the system's functionality, typically containing its own `.api` (controllers), `.application` (services), `.domain` (entities and logic), and `.infrastructure` (repositories) sub-packages:
 - **`.analytics`:** Handles data aggregation, time-series statistics, and the export engine for researchers.
 - **`.auth`:** Manages security configuration, JWT providers, OAuth2 success/failure handlers, and registration logic.
 - **`.classification`:** The core engine managing the image stream, swipe submissions, consensus calculations, and the STARdbi adapter.
 - **`.gamification`:** Orchestrates the rewards system, including point calculations, streak maintenance, and leaderboard management.
 - **`.recipients`:** Manages the logic for grouping users into specific cohorts (Recipient Groups) for targeted tasks.
 - **`.tasks`:** Handles the lifecycle and management of classification assignments.
 - **`.users`:** Manages user profiles, credibility scores, and internal user-related events.
2. **Supporting Infrastructure**

- **.config**: Centralized configuration for the application, including REST clients, async executors, and mock data seeders for development.
- **.eventing**: Contains the messaging infrastructure (e.g., Kafka configuration and publishers) used for decoupled communication between modules.
- **.exception**: Defines the global exception handling strategy and custom API error responses.
- **.model.enums**: Shared enumerations used across multiple domains (e.g., UserRole).

3. Cross-Cutting Concerns

- **.dto**: Data Transfer Objects used to decouple the internal domain models from the external API contracts. This includes `.request` and `.response` sub-packages for various operations like registration and task management.

5.3.2 Data Migrations

The database evolution is managed under `src/main/resources/db/migration`, using Flyway to version the schema from initial setup to specific feature additions like gamification and performance indexing

5.4 Unit Testing

The SwipeLab testing suite is designed to ensure the reliability of the image-labeling pipeline and the accuracy of the gamification engine. We utilize **JUnit 5** and **Mockito** for isolation, alongside **Spring Boot Test** for integrated context verification.

5.4.1 Service Layer Testing (Unit Tests)

Service tests verify business logic in total isolation by mocking all external dependencies (Repositories, Security, and External APIs).

- **Classification Logic**: Tests verify that when a user swipes, the system correctly identifies "trap" images, updates hidden credibility scores, and triggers the appropriate domain events.
- **Gamification Engine**: Unit tests in the gamification module ensure that point multipliers for streaks (e.g., 1.5x for 30+ days) are calculated correctly and that badges are awarded only when JSON-defined criteria are met.
- **Security & Auth**: Tests for AuthenticationService verify that JWT tokens are correctly generated upon login and that unauthorized access to researcher-only endpoints is blocked.

5.4.2 Controller Layer Testing (Web Slice)

We utilize `@WebMvcTest` to verify the REST API contract without starting the full server.

- **Payload Validation**: Ensures that incoming requests, such as `RegisterRequest`, are rejected if fields like email or username are missing or malformed.

- **Mapping Correctness:** Verifies that internal entities are correctly mapped to DTOs (Data Transfer Objects) before being returned to the mobile client to prevent leaking sensitive data.

5.4.3 Event-Driven & Integration Testing

Because the system now uses asynchronous eventing (e.g., Kafka or Spring Events), the testing strategy includes:

- **Asynchronous Processing:** Tests verify that classification submissions correctly trigger background tasks for consensus calculation and leaderboard updates without blocking the user's mobile experience.
- **Persistence Testing:** Integration tests using an H2 in-memory database or Testcontainers ensure that complex SQL queries for analytics and consensus are functionally correct against a real schema.

5.4.4 Test Coverage and Mocking Standards

- **Mockito @Mock and @InjectMocks:** Used across all service tests to simulate repository responses (e.g., `when(userRepository.findByUsername(...)).thenReturn(...)`).
- **Security Mocking:** We use `@WithMockUser` to simulate different roles (USER vs. ADMIN) when testing endpoint protection and dashboard access.

6. User Interface Draft

6.1 User's Interface (Player Mode)

Those screens are intended for a regular users such as volunteers, researchers and anyone who is assigned to a labeling task.

6.1.1 User Login



Login via email & password

Login via Google account

6.1.2 Main User's Page

Logout Button

User Info Button

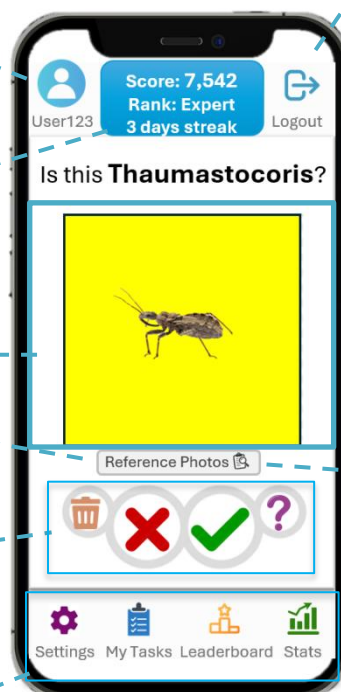
Challenges Button

Swipe Area

Reference Images Button

YES/NO/DON'T KNOW/TRASH

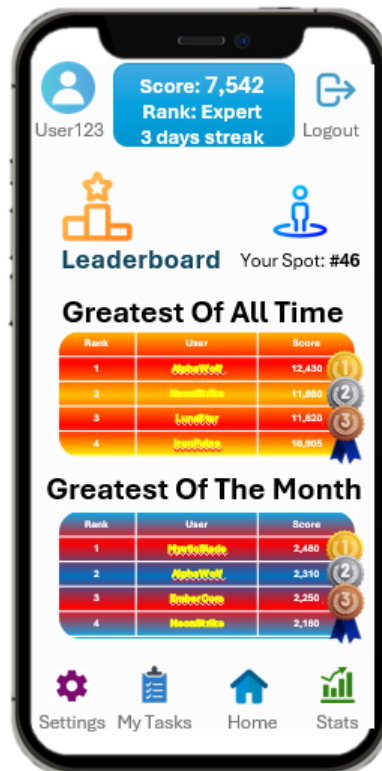
Toolbar Component



6.1.3 User's Task Page



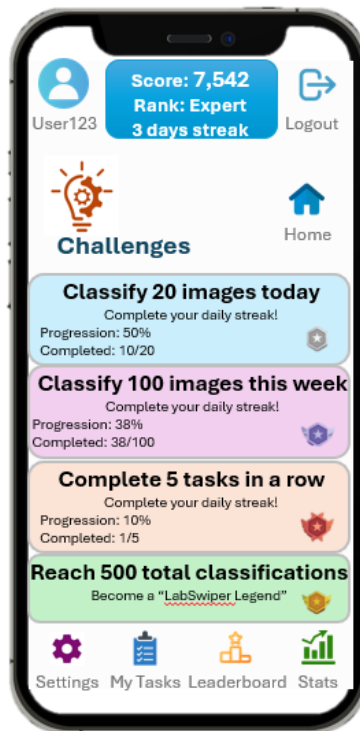
6.1.4 Leaderboard Page



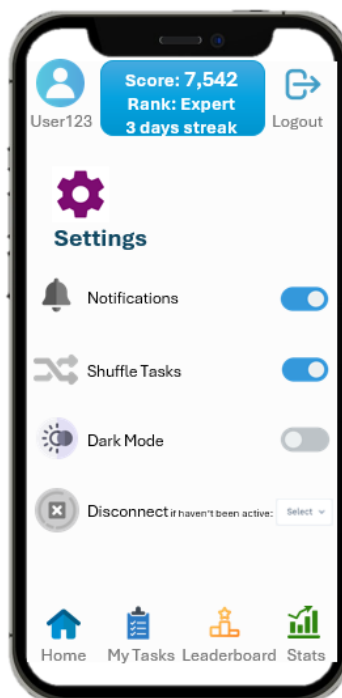
6.1.5 Stats Page (User's Progression)



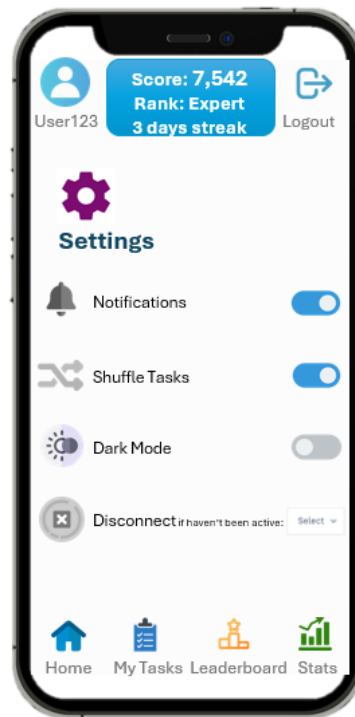
6.1.6 Challenges Page



6.1.7 User's Info Page



6.1.8 Settings Page



6.2 Admin's Interface (Manager Mode)

Those screens are intended for a researcher who wants to create and manage tasks, manage users and analyze the labeling data by the SwipeLab app.

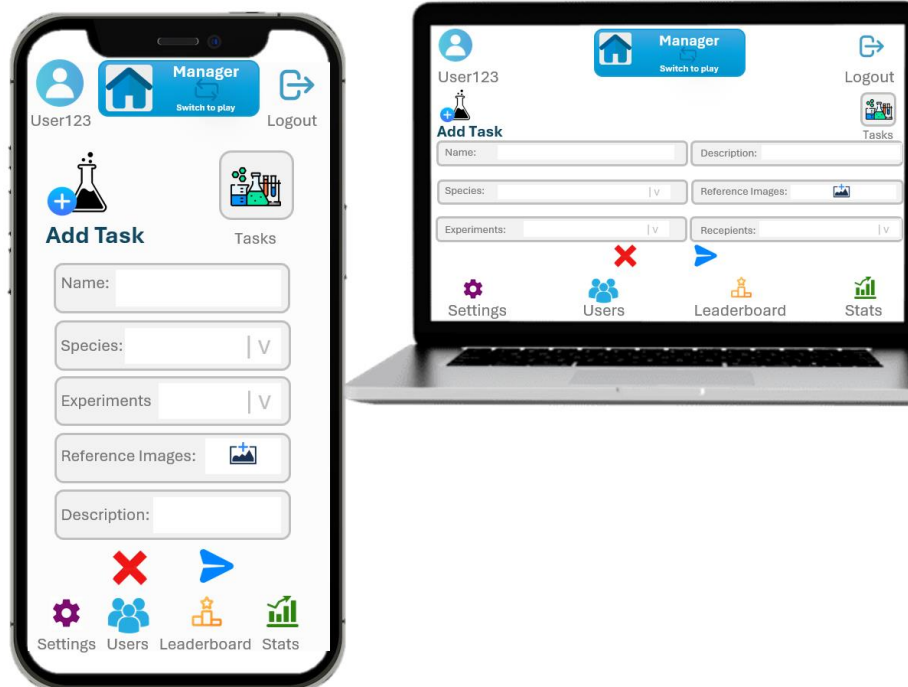
6.2.1 Manger's Dashboard (Main Page)



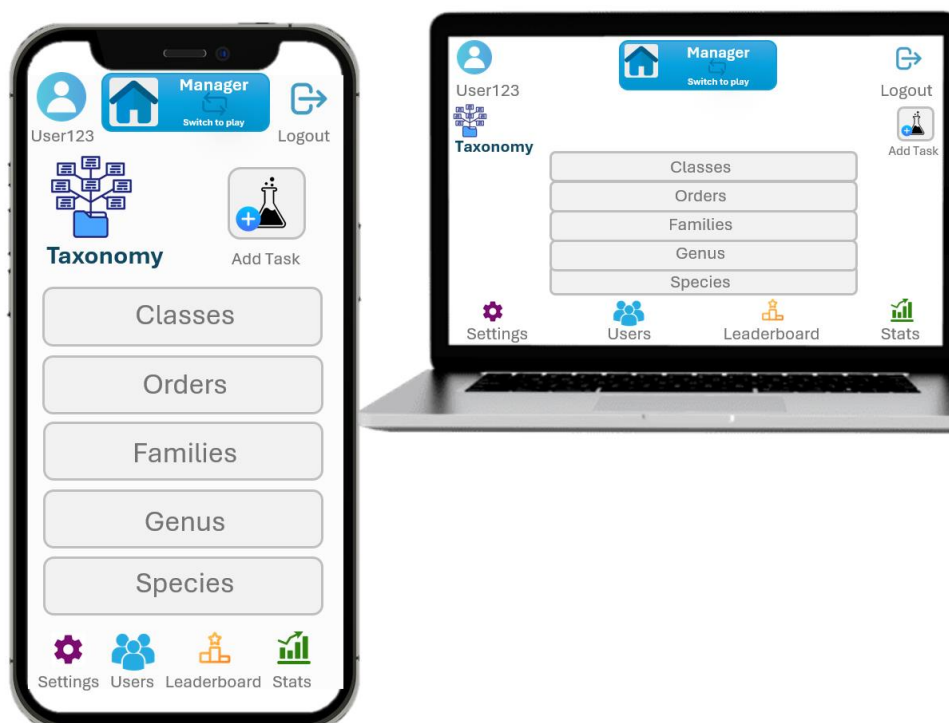
6.2.2 Tasks Management Page



6.2.3 Add Task Page



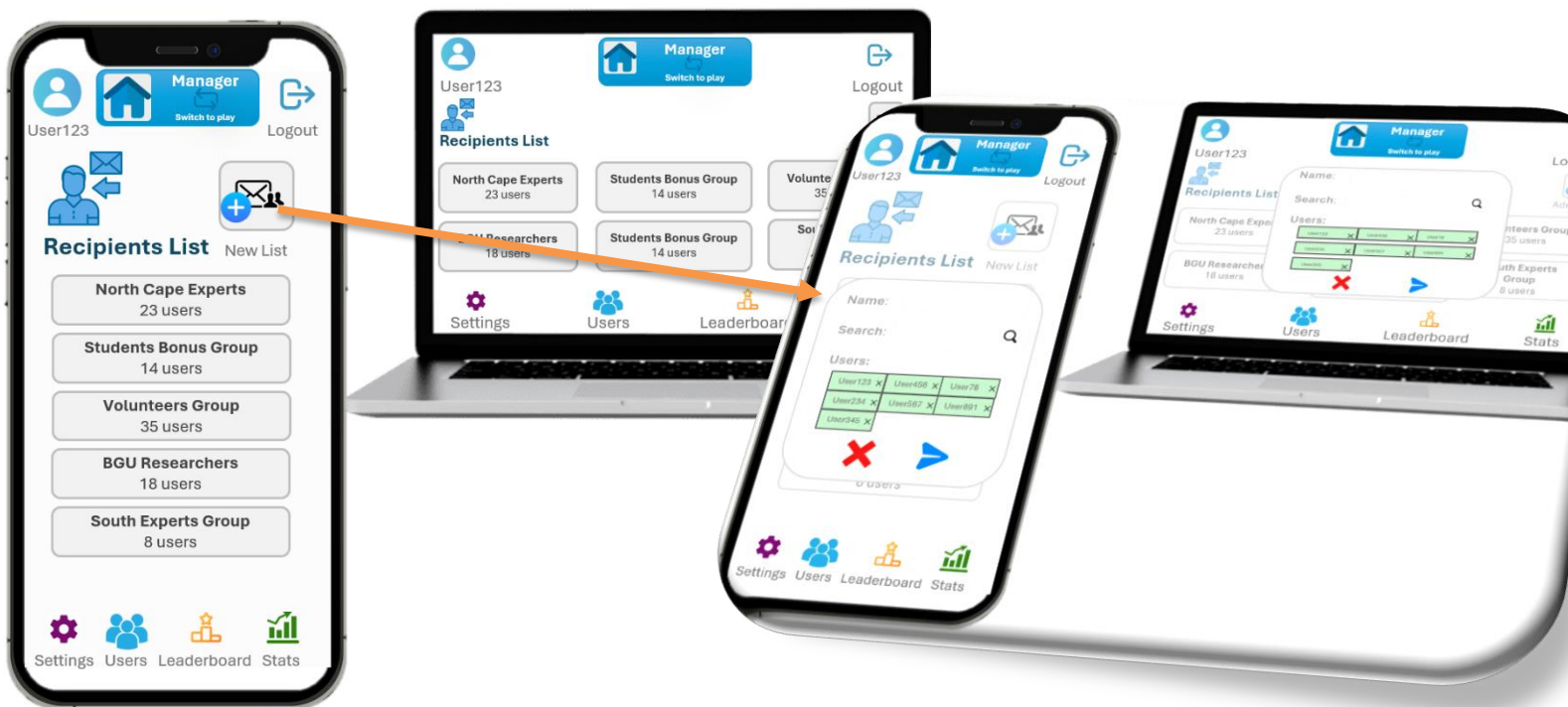
6.2.4 Taxonomy Info Page



6.2.5 Users Management Page



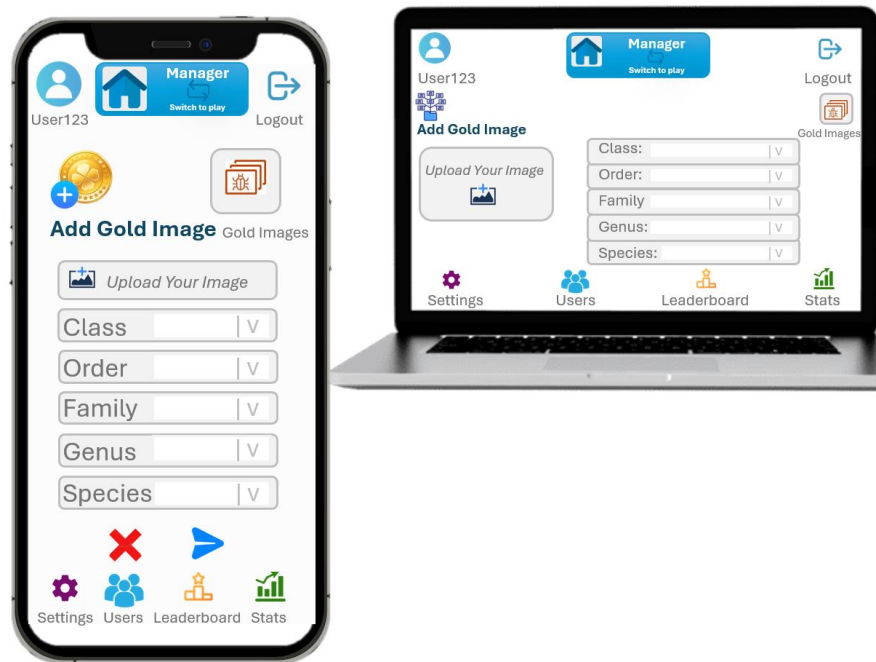
6.2.6 Recipients Management Page



6.2.7 Gold Images Management Page



6.2.8 Add Gold Image Page



7. Testing

SwipeLab Testing Strategy & Documentation

1. Overview

The SwipeLab project employs a robust, full-stack testing strategy. Because the application is divided into a Java/Spring Boot backend and a React Native frontend, the testing architecture is split into two primary domains:

- **Backend:** A heavy emphasis on Unit and Integration testing to ensure data integrity, complex gamification logic, and cross-domain eventing.
 - **Frontend:** End-to-End (E2E) testing to ensure user flows (like swiping, viewing stats, and admin management) work flawlessly from a user's perspective.
-

2. Backend Testing (Java / Spring Boot)

2.1 Tools & Frameworks

While the exact dependencies are managed in pom.xml, the standard SwipeLab backend testing stack includes:

- **JUnit 5:** The core testing framework for test execution and lifecycle management.
- **Mockito:** Used for mocking dependencies (Repositories, external services) to isolate unit tests.
- **Spring Boot Test:** Provides context loading and utilities like MockMvc for API endpoint testing.
- **H2 / Testcontainers (Inferred):** Managed via application-test.properties and application-integration.properties for testing against temporary databases without affecting production.

2.2 Package Structure

The backend test suite strictly mirrors the **Domain-Driven Design (DDD)** of the src/main/java folder. Tests are grouped by domain (auth, classification, gamification, recipients, tasks, users), and further subdivided by architectural layers:

- api/ (Controllers)
- application/ (Services, Event Listeners)
- domain/ (Core logic, Calculators, Mappers)
- infrastructure/ (Security, Configuration)

2.3 Types of Backend Tests

A. Domain & Utility Unit Tests (Isolated)

These tests verify core business logic without loading the Spring Context. They are lightning-fast and mock all external dependencies.

- **Calculators/Logic:** Tests like CredibilityCalculatorTest.java verify the math behind user scoring.
- **Mappers:** Tests like AuthMapperTest.java and TaskMapperTest.java ensure Data Transfer Objects (DTOs) map correctly to Entities.

B. Application Service Tests

These test the business use cases. Dependencies like repositories are mocked to ensure the service logic (e.g., fetching images, calculating streaks) behaves correctly.

- *Examples:* ClassificationServiceTest.java, GamificationOrchestratorServiceTest.java, TaskDistributionServiceTest.java.

C. API / Controller Tests

These tests use MockMvc to simulate HTTP requests and verify routing, status codes, JSON serialization, and input validation without needing a running server.

- *Examples:* AuthControllerTest.java, ClassificationControllerTest.java, GamificationControllerTest.java.

D. Integration Tests

These are crucial for SwipeLab, as the app relies on event-driven architecture (e.g., classifying an image triggers gamification updates). These tests load a larger Spring context to verify that different domains talk to each other correctly.

- *Examples:* * ClassificationToGamificationIntegrationTest.java: Ensures a swipe results in points/badges.
 - FraudDetectionFlowIntegrationTest.java: Ensures bad classifications properly flag users.
 - GamificationSyncIntegrationTest.java: Verifies end-to-end sync logic.

3. Frontend Testing (React Native / Expo)

3.1 Tools & Frameworks

- **Playwright:** The primary framework used for Frontend testing, configured via frontend/playwright.config.ts.
- *(Note: The current structure heavily favors E2E testing over isolated component testing like Jest, ensuring the app works holistically).*

3.2 Types of Frontend Tests

The frontend test suite (located in frontend/tests/) focuses entirely on **End-to-End (E2E) User Journeys**. These tests launch the application and interact with it exactly as a real human would (tapping buttons, filling forms, swiping).

They are divided into three main categories:

A. Shared/Core Flows

- auth.spec.ts: Tests login, registration, and logout flows.
- settings.spec.ts: Tests profile updates and app configuration.

B. User "Swiper" Flows

Tests that simulate the primary target audience interacting with the gamified app.

- user-swipe.spec.ts: The core functionality—fetching an image, swiping, and submitting a classification.
- user-stats.spec.ts: Verifying the dashboard, credibility scores, and history.
- user-challenges.spec.ts: Ensuring daily/weekly challenges update correctly.
- my-collection.spec.ts: Testing the gallery of classified images/badges.

C. Admin Flows

Tests ensuring laboratory admins can manage the crowdsourcing data.

- admin-dashboard.spec.ts: Tests the high-level metrics view.
- admin-tasks.spec.ts: Tests the creation, editing, and distribution of image tasks.
- admin-recipients.spec.ts: Tests the management of target user groups.

4. Testing Conventions & Guidelines

To maintain code quality, the team follows these conventions:

1. Naming Conventions:

- **Backend:** All test classes must end with Test.java (e.g., UserServiceTest.java).
- **Frontend:** All Playwright tests must end with .spec.ts (e.g., admin-tasks.spec.ts).

2. **Environment Separation:**

- The backend utilizes application-test.properties for fast, in-memory unit tests, and application-integration.properties for heavier context loads.

3. **Mocking External Systems:**

- Any third-party integration (like the StardbiClient and EmailService) must be mocked during standard unit tests to prevent network failures from causing flaky tests.

4. **Test Data:**

- Rely on factories or the MockDataSeeder logic when generating test data to avoid brittle tests dependent on hardcoded database states.

5. **CI/CD Guardrails:**

- (*Best Practice based on Git workflow*): All PRs targeting the main branch must pass the full suite of backend Unit Tests and frontend Playwright specs before they can be merged.